

DOCUMENTACIÓN TÉCNICA

Sistema de Recaudo de Préstamos de Dinero

Autor: **Camilo Andrés Castro Acosta**

Tecnología: Java 17 · Spring Boot 3 · MongoDB · JWT

Fecha: 03 de June de 2026

Documento de Arquitectura, Diseño y Código Fuente

TABLA DE CONTENIDO

1. Descripción General del Sistema
2. Requisitos Funcionales
3. Requisitos No Funcionales
4. Arquitectura del Sistema
5. Patrones de Diseño Aplicados
6. Estructura de Paquetes
7. Capa de Modelo / Dominio — Clases y Métodos
8. Herencia y Polimorfismo
9. Capa DTO (Data Transfer Objects)
10. Capa de Repositorio
11. Capa de Servicios — Interfaces e Implementaciones
12. Capa de Controladores REST
13. Seguridad — JWT y Spring Security
14. Manejo de Excepciones
15. Módulo de Reportes
16. Endpoints REST — Resumen
17. Tecnologías y Dependencias
18. Conclusiones

1. DESCRIPCIÓN GENERAL DEL SISTEMA

El **Sistema de Recaudo de Préstamos de Dinero** es una aplicación de back-end construida sobre **Spring Boot 3 / Java 17** con base de datos **MongoDB**. Gestiona todo el ciclo de vida de un préstamo: desde la solicitud y aprobación, pasando por la generación automática de cuotas con fórmula francesa, hasta el registro de pagos, cálculo de mora y generación de reportes exportables en PDF y Excel.

Actores del sistema:

Actor	Responsabilidad
Cliente	Solicita préstamos, consulta su deuda e historial de pagos a través de la app web o móvil.
Recaudador	Registra pagos, consulta deudas e imprime recibos de pago.
Administrador	Gestiona usuarios, aprueba/rechaza préstamos, configura parámetros y genera reportes.

2. REQUISITOS FUNCIONALES

RF-01 al RF-06 — Gestión de Clientes

ID	Descripción
RF-01	Registrar un cliente con datos personales (nombres, apellidos, documento, teléfono, dirección, correo).
RF-02	Consultar un cliente por su ID interno de MongoDB.
RF-03	Listar todos los clientes activos del sistema.
RF-04	Actualizar datos de contacto de un cliente existente.
RF-05	Eliminar (dar de baja) un cliente del sistema.
RF-06	Validar que no existan documentos de identidad duplicados al registrar.

RF-07 al RF-14 — Gestión de Préstamos

ID	Descripción
RF-07	Solicitar un nuevo préstamo indicando clienteld, monto, plazo en meses y tasa de interés.
RF-08	Aprobar un préstamo en estado PENDIENTE, generando automáticamente las cuotas.
RF-09	Rechazar un préstamo en estado PENDIENTE.
RF-10	Consultar un préstamo por ID.
RF-11	Listar todos los préstamos o filtrarlos por cliente.
RF-12	Calcular el valor de cuota usando la fórmula francesa (cuota fija con amortización).
RF-13	Calcular mora sobre cuotas vencidas (0.5% diario sobre saldo).
RF-14	Actualizar automáticamente el estado del préstamo a CANCELADO cuando saldo pendiente sea cero.

RF-15 al RF-19 — Gestión de Pagos

ID	Descripción
RF-15	Registrar un pago para un préstamo activo, distribuyendo el valor en las cuotas pendientes (pago parcial o total).
RF-16	Generar número de comprobante único por pago.
RF-17	Consultar todos los pagos de un préstamo.
RF-18	Calcular el total de abonos realizados sobre un préstamo.
RF-19	Validar que el valor del pago sea mayor a cero y que el préstamo esté activo.

RF-20 al RF-23 — Gestión de Usuarios y Roles

ID	Descripción
RF-20	Crear usuarios del sistema (Administrador, Recaudador, ClienteUsuario) con contraseña cifrada.
RF-21	Asignar o cambiar el rol de un usuario.
RF-22	Activar/desactivar usuarios del sistema.
RF-23	Consultar y listar usuarios registrados.

RF-24 al RF-26 — Autenticación

ID	Descripción
RF-24	Autenticar usuarios mediante email y contraseña, retornando JWT de acceso y refresh.
RF-25	Renovar el token de acceso usando el refresh token.
RF-26	Cambiar contraseña validando la actual e ingresando la nueva.

RF-27 al RF-30 — Reportes y Parámetros

ID	Descripción
RF-27	Generar reporte de cartera total (monto, saldo pendiente, cantidad de préstamos).
RF-28	Generar reporte de mora con cuotas vencidas por préstamo y cliente.
RF-29	Exportar reportes en formato PDF (iText) y Excel (Apache POI).
RF-30	Gestionar parámetros de configuración del sistema (clave/valor persistidos en MongoDB).

3. REQUISITOS NO FUNCIONALES

ID	Categoría	Descripción
RNF-01	Seguridad	Todas las contraseñas se almacenan con hash BCrypt. Las rutas protegidas requieren JWT válido. Roles controlados por Spring Security a nivel de endpoint.
RNF-02	Autenticación sin estado	La API es completamente stateless: no usa sesiones HTTP. Cada petición porta su propio token Bearer.
RNF-03	Escalabilidad	MongoDB soporta replica sets, permitiendo escalar la capa de datos horizontalmente. La aplicación puede desplegarse en múltiples instancias detrás de un balanceador.
RNF-04	Disponibilidad	El servidor de archivos/backups funciona de forma independiente al servidor de aplicaciones para evitar puntos únicos de fallo.
RNF-05	Mantenibilidad	Arquitectura en capas estrictas (Controller → Service → Repository). Cada capa depende solo de abstracciones (interfaces), facilitando cambios.
RNF-06	Precisión financiera	Todos los valores monetarios usan BigDecimal con escala 2 y RoundingMode.HALF_UP, evitando errores de punto flotante.
RNF-07	Portabilidad del cliente	La API REST con CORS habilitado permite consumirla desde aplicaciones web (React, Angular) y móviles (Android/iOS).
RNF-08	Rendimiento	Las consultas MongoDB se filtran por campos indexados (clienteId, estado). Los reportes se generan en memoria y se retornan como bytes descargables.
RNF-09	Trazabilidad	Cada usuario registra fechaCreacion y ultimoAcceso. Los pagos almacenan fecha, metodoPago y referencia.
RNF-10	Usabilidad de la API	Respuestas HTTP con códigos semánticos (201 Created, 204 No Content, 404 Not Found). Excepciones centralizadas con mensajes descriptivos en español.

4. ARQUITECTURA DEL SISTEMA

El sistema sigue la arquitectura de **capas (Layered Architecture)** combinada con el patrón **MVC** adaptado a APIs REST. Las cinco capas son:

Capa	Responsabilidad
Controlador (Controller)	Expone los endpoints REST. Recibe la petición HTTP, delega al servicio y retorna ResponseEntity con el código adecuado. No contiene lógica de negocio.
DTO (Transfer Objects)	Objetos planos (POJO) usados para transportar datos entre cliente y servidor. Desacoplan el modelo de dominio de la representación externa.
Servicio (Service / Business Logic)	Contiene toda la lógica de negocio. Orquesta operaciones, aplica validaciones, lanza excepciones de negocio y coordina llamadas al repositorio.
Repositorio (Repository / Data Access)	Interfaces que extienden MongoRepository. Encapsulan el acceso a MongoDB. Spring Data genera las implementaciones en tiempo de ejecución.
Modelo / Dominio (Domain)	Entidades de negocio anotadas con @Document. Contienen atributos y métodos de dominio (cálculos, validaciones propias). No dependen de capas superiores.

Vista de Despliegue (Infraestructura):

Nodo	Descripción
Dispositivo Cliente	Navegador web o App móvil (Android/iOS). Se comunica con el servidor de aplicaciones mediante HTTPS.
Servidor de Aplicaciones	Ejecuta Spring Boot (Java) con la API REST y los Servicios de Negocio. Punto central del sistema.
Servidor de Base de Datos	MongoDB Server configurado con Replica Set. Conectado al servidor de aplicaciones por TCP/IP.
Servidor de Archivos	Almacena backups, reportes exportados y archivos adjuntos. Comunicación dashed (no crítica) con el servidor de aplicaciones.

5. PATRONES DE DISEÑO APLICADOS

Patrón	Dónde se aplica	Clases involucradas	Justificación
Repository Pattern	Capa de repositorio	ClienteRepository, PrestamoRepository, PagoRepository, UsuarioRepository, RolRepository, ParametroRepository	Abstrae el acceso a MongoDB. Las interfaces declaran operaciones y Spring Data genera la implementación. Permite sustituir la base de datos sin cambiar la lógica de negocio.
Service Layer Pattern	Capa de servicio	ClienteService, PrestamoService, PagoService, CuotaService, AuthService, UsuarioService, ReporteService, ParametroService	Centraliza la lógica de negocio en servicios cohesivos. Cada servicio expone una interfaz; la implementación (Impl) está desacoplada, facilitando pruebas unitarias con mocks.
DTO Pattern	Capa DTO	ClienteDTO, PrestamoDTO, PagoDTO, CuotaDTO, UsuarioDTO, LoginDTO, ReporteDTO, ParametroDTO	Separa el modelo interno de la representación de la API. Evita exponer campos sensibles (ej.: passwordHash) y permite validar la entrada antes de pasarla a la capa de dominio.
Dependency Injection (IoC)	Toda la aplicación	@Autowired en todos los servicios y controladores	Spring gestiona el ciclo de vida de los beans. Los controladores no crean servicios directamente; los reciben inyectados, reduciendo acoplamiento.
Template Method (implícito)	Capa de modelo	Usuario → Administrador, Recaudador, ClienteUsuario	La clase base Usuario define la estructura y los métodos comunes (autenticar, cambiarContraseña). Las subclases añaden comportamiento especializado sin duplicar código.
Chain of Responsibility	Capa de seguridad	JwtAuthenticationFilter → UsernamePasswordAuthenticationFilter → SecurityFilterChain	Spring Security encadena filtros. JwtAuthenticationFilter intercepta cada petición, extrae el token, valida y carga el contexto de seguridad antes de delegar al siguiente filtro.
Strategy (a través de interfaces)	Capa de servicio	ReporteService: exportarPDF vs exportarExcel	El servicio de reportes permite exportar en diferentes formatos (PDF con iText, Excel con Apache POI) sin que el controlador conozca la librería usada.
Embedded Documents	Modelo MongoDB	Cuota y Pago embebidos dentro de Prestamo	En lugar de colecciones separadas, Cuota y Pago son documentos embebidos en Prestamo. Esto mejora el rendimiento de lectura y mantiene la consistencia del documento de negocio.

6. ESTRUCTURA DE PAQUETES

Paquete	Contenido / Responsabilidad
com.prestamos	Clase principal SistemaPrestamosApplication (@SpringBootApplication)
com.prestamos.model	Entidades de dominio: Cliente, Prestamo, Cuota, Pago, Usuario, Administrador, Recaudador, ClienteUsuario, Rol, Parametro
com.prestamos.dto	Objetos de transferencia: ClienteDTO, PrestamoDTO, CuotaDTO, PagoDTO, UsuarioDTO, LoginDTO, ReporteDTO, ParametroDTO
com.prestamos.repository	Interfaces MongoRepository: ClienteRepository, PrestamoRepository, UsuarioRepository, RolRepository, ParametroRepository
com.prestamos.service	Interfaces de servicio: ClienteService, PrestamoService, PagoService, CuotaService, UsuarioService, AuthService, ReporteService, ParametroService
com.prestamos.service.impl	Implementaciones @Service de cada interfaz (sufijo Impl)
com.prestamos.controller	Controllers REST @RestController: 8 controladores (uno por dominio)
com.prestamos.security	CustomUserDetailsService — carga UserDetails desde MongoDB para Spring Security
com.prestamos.security.jwt	JwtUtil (generación/validación de tokens), JwtAuthenticationFilter (filtro HTTP)
com.prestamos.config	SecurityConfig (cadena de filtros), DataInitializer (carga datos iniciales al arrancar)
com.prestamos.exception	GlobalExceptionHandler (@RestControllerAdvice), NegocioException, RecursoNoEncontradoException

7. CAPA DE MODELO / DOMINIO

Todas las clases de dominio están en el paquete **com.prestamos.modelo**. Se usan las anotaciones de Lombok (`@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`) para reducir código boilerplate y las anotaciones de Spring Data MongoDB (`@Document`, `@Id`, `@DBRef`) para el mapeo objeto-documento.

7.1 Cliente

Representa a la persona natural que solicita un préstamo. Se persiste en la colección **clientes**.

Método	Descripción
<code>getIdCliente() : int</code>	Retorna el hashCode del ID MongoDB como entero (compatibilidad con diagrama UML).
<code>setIdCliente(int) : void</code>	Asigna el ID a partir de un entero convirtiéndolo a String.
<code>getNombre() : String</code>	Concatena nombres + apellidos en una cadena.
<code>setNombre(String) : void</code>	Divide la cadena en nombres y apellidos por el primer espacio.
<code>consultar() : Cliente</code>	Retorna la instancia actual (útil para encadenamiento).
<code>actualizar() : void</code>	Marcador semántico; la lógica real reside en <code>ClienteServiceImpl.actualizarCliente()</code> .
<code>toString() : String</code>	Representación textual con id, nombres y documento.

7.2 Prestamo

Entidad central del sistema. Persiste en la colección **prestamos**. Contiene listas embebidas de Cuota y Pago.

Método	Descripción
<code>calcularCuotas() : BigDecimal</code>	Aplica la fórmula francesa (cuota fija): $C = M * i * (1+i)^n / ((1+i)^n - 1)$. Usa <code>BigDecimal.pow()</code> y <code>RoundingMode.HALF_UP</code> .
<code>calcularInteres() : BigDecimal</code>	Calcula el interés simple mensual: <code>monto * tasaInteres / 100</code> .
<code>calcularSaldoPendiente() : BigDecimal</code>	Suma los saldos de todas las cuotas que no tienen estado PAGADA usando Stream API.
<code>aprobar() : void</code>	Cambia estado a APROBADO e invoca <code>generarCuotas()</code> .
<code>generarCuotas() : void</code>	Crea la lista de objetos Cuota con fecha de vencimiento mensual incremental usando Calendar. Inicializa saldo = valor de cuota.
<code>consultar() : Prestamo</code>	Retorna la instancia actual.

7.3 Cuota

Documento embebido dentro de Prestamo. Modela cada cuota del plan de pago. Estados posibles: PENDIENTE, PAGADA, VENCIDA.

Método	Descripción
<code>verificarMora() : boolean</code>	Retorna true si el estado es PENDIENTE y la fechaVencimiento ya pasó (<code>Date.after()</code>).

pagaoParcial(BigDecimal) : BigDecimal	Aplica un monto de pago a la cuota. Si el pago supera el saldo, la marca PAGADA y retorna el excedente para aplicar a la siguiente cuota.
getSaldoDeCuota() : BigDecimal	Retorna el saldo actual o BigDecimal.ZERO si es nulo.
getInteres() : BigDecimal	Retorna BigDecimal.ZERO (el interés está implícito en el valor de la cuota).

7.4 Pago

Documento embebido en Prestamo. Registra cada transacción de pago realizada sobre el préstamo.

Método	Descripción
generarComprobante() : String	Genera el número de comprobante con prefijo COMP- concatenado con el ObjectId.
autenticar() : boolean	Verifica que el estado del pago no sea nulo ni vacío.

7.5 Usuario

Clase base de la jerarquía de usuarios. Se persiste en la colección **usuarios**. Tiene una referencia @DBRef a Rol, que permite la resolución perezosa del rol desde su colección.

Método	Descripción
autenticar(String) : boolean	Compara el password ingresado con el almacenado (en bruto; la comparación cifrada la realiza Spring Security con BCrypt).
cambiarContrasena(String) : boolean	Valida que la nueva contraseña no sea nula/vacía y la asigna.

7.6 Rol

Entidad que define los perfiles de acceso del sistema (ADMINISTRADOR, RECAUDADOR, CLIENTEUSUARIO). Almacena una lista de permisos como Strings.

7.7 Parametro

Entidad clave-valor para configuración dinámica del sistema (ej.: tasa de interés por defecto, límites de préstamo). Persiste en la colección **parametros**.

8. HERENCIA Y POLIMORFISMO

El sistema implementa una jerarquía de usuarios mediante **herencia simple** en Java. La clase base es **Usuario**; las subclases añaden comportamientos específicos de cada rol.

Clase	Relación	Atributos propios	Métodos propios
Usuario (base)	—	id, nombre, username, password, email, estado, fechaCreacion, ultimoAcceso, rol	autenticar(String), cambiarContrasena(String), getIdUsuario(), toString()
Administrador	extends Usuario	Ningún campo adicional	gestionarUsuarios(), gestionarSistema(), verReportes()
Recaudador	extends Usuario	Ningún campo adicional	registrarPago(), consultarDeuda(), imprimirRecibo()
ClienteUsuario	extends Usuario	Ningún campo adicional	solicitarPrestamo(), verHistorialPagos()

Anotaciones de Lombok en herencia:

Se usa **@EqualsAndHashCode(callSuper = true)** en cada subclase para que equals/hashCode incluyan los campos de la clase padre.

@Document(collection = "usuarios") se repite en subclases, permitiendo polimorfismo en la misma colección MongoDB (patrón Single Collection Inheritance).

Polimorfismo a través de interfaces de servicio:

Cada dominio tiene una interfaz de servicio y al menos una implementación. Spring inyecta la implementación correcta en tiempo de ejecución mediante el contenedor IoC. Esto es polimorfismo de subtipo: el controlador solo conoce la interfaz, no la clase concreta.

Interfaz	Implementación	Beneficio del polimorfismo
ClienteService	ClienteServiceImpl	Controlador inyecta ClienteService; puede sustituirse por otra impl sin modificar el controlador.
ReporteService	ReporteServiceImpl	Exporta PDF (iText) o Excel (Apache POI) según el método llamado; ambos implementan la misma interfaz.
AuthService	AuthServiceImpl	La autenticación JWT puede reemplazarse por OAuth2 implementando la misma interfaz.

9. CAPA DTO (DATA TRANSFER OBJECTS)

DTO	Campos principales	Uso
ClienteDTO	nombres, apellidos, documento, telefono, direccion, correo, estado	Entrada para POST/PUT /api/clientes
PrestamoDTO	clienteId, monto, plazoMeses, interesAnual, fechaSolicitud, estado	Entrada para solicitar un préstamo
CuotaDTO	numero, valor, fechaVencimiento, estado, saldo, pagado	Representación de cuota para respuestas
PagoDTO	prestamold, cuotald, valor, fechaPago, estado, metodoPago, referencia	Entrada para registrar un pago
UsuarioDTO	id, nombres, email, username, rolId, estado	Crear/actualizar usuarios del sistema
LoginDTO	email, password	Autenticación: obtener tokens JWT
ReporteDTO	tipoReporte, fechaInicio, fechaFin, filtros	Parámetros para generación de reportes
ParametroDTO	id, clave, valor, descripcion	Gestión de configuración del sistema

10. CAPA DE REPOSITORIO (ACCESO A DATOS)

Todos los repositorios extienden **MongoRepository<T, String>**. Spring Data MongoDB genera la implementación automáticamente en tiempo de ejecución basándose en el nombre de los métodos.

Repositorio	Entidad	Métodos principales
ClienteRepository	Cliente	save(), findById(), findAll(), deleteById(), existsById(), existsByDocumento(String)
PrestamoRepository	Prestamo	save(), findById(), findAll(), findByClienteld(String), countByEstado(String)
UsuarioRepository	Usuario	save(), findById(), findAll(), findByEmail(String), findByUsername(String), existsByEmail(String), existsByUsername(String)
RolRepository	Rol	save(), findById(), findAll()
ParametroRepository	Parametro	save(), findById(), findAll(), findByClave(String)

Nota sobre documentos embebidos:

Cuota y Pago NO tienen repositorio propio porque están embebidos dentro del documento Prestamo. Su acceso y modificación se realiza cargando el Prestamo padre y actualizando la lista.

11. CAPA DE SERVICIOS

ClienteService / ClienteServiceImpl

Método	Descripción
registrarCliente(ClienteDTO) : Cliente	Verifica que el documento no esté duplicado (existsByDocumento). Mapea DTO → entidad, asigna fechaRegistro=now y estado=ACTIVO. Persiste con save().
actualizarCliente(String, ClienteDTO) : Cliente	Carga el cliente por ID (lanza RecursoNoEncontradoException si no existe). Actualiza campos de contacto y persiste.
consultarCliente(String) : Cliente	Busca por ID; lanza RecursoNoEncontradoException si no se encuentra.
consultarClientes() : List	Retorna todos los clientes de la colección mediante findAll().
eliminarCliente(String) : void	Verifica existencia y luego llama deleteById().
validarCliente(String) : boolean	Retorna existsById() — útil antes de crear un préstamo.

PrestamoService / PrestamoServiceImpl

Método	Descripción
solicitarPrestamo(PrestamoDTO) : Prestamo	Valida que el cliente exista, que el monto sea >0 y el plazo >0. Crea el Prestamo con estado PENDIENTE y lo persiste.
aprobarPrestamo(String) : Prestamo	Carga el préstamo, verifica estado PENDIENTE, llama prestamo.aprobar() que genera cuotas, y persiste.
rechazarPrestamo(String) : Prestamo	Cambia estado a RECHAZADO y persiste.
calcularInteres(String) : void	Invoca prestamo.calcularInteres() y actualiza tasaInteres en DB.
calcularMora(String) : void	Itera cuotas vencidas, calcula mora diaria (0.5% sobre saldo × días), agrega mora al saldo y marca cuota como VENCIDA.
marcarCuotaPagada(String, int) : void	Localiza la cuota por número, la marca PAGADA, saldo=0, recalcula saldoPendiente y si queda en cero marca préstamo como CANCELADO.

PagoService / PagoServiceImpl

Método	Descripción
registrarPago(PagoDTO) : Prestamo	Valida el pago, carga el préstamo, itera cuotas pendientes aplicando pago parcial mediante Cuota.pagarParcial(). Crea objeto Pago con ID ObjectId, agrega a la lista de pagos, recalcula saldoPendiente y persiste.
consultarPagosPorPrestamo(String) : List	Retorna la lista de pagos embebida en el Prestamo.
generarComprobante(String, String) : String	Localiza el Pago por ID dentro del Prestamo y llama pago.generarComprobante().
validarPago(PagoDTO) : void	Lanza NegocioException si valor es <=0 o prestamold es nulo/vacío.
calcularAbono(String) : BigDecimal	Suma todos los valores de los pagos del préstamo usando Stream.reduce().

CuotaService / CuotaServiceImpl

Método	Descripción
generarCuotas(String) : List	Carga el préstamo, invoca prestamo.generarCuotas() y persiste el préstamo actualizado.
consultarCuotas(String) : List	Retorna la lista de cuotas embebidas en el Prestamo.
calcularSaldoCuota(String, int) : BigDecimal	Filtra la cuota por número dentro del préstamo y retorna su saldo.

UsuarioService / UsuarioServiceImpl

Método	Descripción
crearUsuario(UsuarioDTO) : Usuario	Verifica unicidad de email y username. Crea usuario con password temporal 123456 cifrado con BCrypt. Asigna rol si se especifica rolId.
actualizarUsuario(String, UsuarioDTO) : Usuario	Actualiza nombre y email del usuario.
cambiarEstado(String, boolean) : void	Activa o desactiva el usuario (soft delete).
asignarRol(String, String) : void	Busca usuario y rol, asigna rol al usuario y persiste.

AuthService / AuthServiceImpl

Método	Descripción
login(LoginDTO) : Map	Autentica con AuthenticationManager (Spring Security). Genera accessToken y refreshToken con JwtUtil. Actualiza ultimoAcceso del usuario en DB.
refreshToken(String) : Map	Extrae username del refreshToken, valida con JwtUtil.validateToken() y genera nuevo accessToken.
cambiarPassword(String,String,String) : boolean	Verifica contraseña actual con BCrypt.matches(), cifra la nueva con BCrypt.encode() y persiste.
logout(String) : void	Stateless: no hay sesión que invalidar; el cliente descarta el token.

12. CAPA DE CONTROLADORES REST

Los controladores reciben las peticiones HTTP, delegan al servicio y retornan **ResponseEntity** con el código HTTP apropiado. No contienen lógica de negocio.

Controlador	Base URL	Endpoints
AuthController	/api/auth	POST /login · GET /logout · POST /refresh · PUT /cambiar-password · DELETE /fin
ClienteController	/api/clientes	POST · GET · GET /{id} · PUT /{id} · DELETE /{id}
PrestamoController	/api/prestamos	POST · GET · GET /{id} · GET /cliente/{cld} · PUT /{id}/aprobar · PUT /{id}/rechazar · DELETE /{id}
PagoController	/api/pagos	POST · GET /prestamo/{pId} · GET /prestamo/{pId}/comprobante/{pagold} · PUT · DELETE /{pId}/{pagold}
CuotaController	/api/cuotas	GET /prestamo/{prestamold}
UsuarioController	/api/usuarios	POST · GET · GET /{id} · PUT /{id} · PUT /{id}/estado · PUT /{id}/rol · DELETE /{id}
ReporteController	/api/reportes	GET /cartera · GET /mora · GET /pagos · GET /exportar/pdf · GET /exportar/excel
ParametroController	/api/parametros	GET · GET /{clave} · PUT /{id}

13. SEGURIDAD — JWT y SPRING SECURITY

13.1 Flujo de Autenticación

Paso	Descripción
1	Cliente envía POST /api/auth/login con {email, password}.
2	AuthController delega a AuthServiceImpl.login().
3	AuthServiceImpl invoca AuthenticationManager.authenticate() que usa CustomUserDetailsService para cargar el usuario y BCryptPasswordEncoder para verificar la contraseña.
4	Si es correcto, JwtUtil genera accessToken (HS256, 1h) y refreshToken (HS256, 7d).
5	Se actualiza el campo ultimoAcceso del usuario en MongoDB.
6	Se retorna {token, refreshToken, username} al cliente.
7	El cliente incluye Authorization: Bearer token en cada petición subsiguiente.
8	JwtAuthenticationFilter intercepta cada petición, extrae el token, valida firma y expiración, y carga SecurityContext.

13.2 Control de Acceso por Rol

Endpoint	Roles autorizados	Nota
/api/auth/**	Público	Sin autenticación — login y refresh
/api/clientes/**	ADMINISTRADOR, RECAUDADOR	Solo usuarios con estos roles
/api/prestamos/**	ADMINISTRADOR, RECAUDADOR, CLIENTEUSUARIO	Los tres roles
/api/pagos/**	ADMINISTRADOR, RECAUDADOR	Gestión de pagos
/api/cuotas/**	ADMINISTRADOR, RECAUDADOR, CLIENTEUSUARIO	Consulta de cuotas
/api/usuarios/**	ADMINISTRADOR	Solo el administrador
/api/reportes/**	ADMINISTRADOR	Solo el administrador
/api/parametros/**	ADMINISTRADOR	Solo el administrador

13.3 JwtUtil — Métodos clave

Método	Descripción
generateToken(UserDetails)	Genera accessToken con expiración configurada en jwt.expiration (application.properties).
generateRefreshToken(UserDetails)	Genera refreshToken con expiración jwt.refresh-expiration.
validateToken(String, UserDetails)	Verifica que el username del token coincide con el UserDetails y que el token no ha expirado.
extractUsername(String)	Extrae el subject (email) del payload del JWT usando JWT parser.
isTokenExpired(String)	Compara la fecha de expiración del token con la fecha actual.

14. MANEJO DE EXCEPCIONES

El sistema centraliza el manejo de excepciones en **GlobalExceptionHandler** anotado con **@RestControllerAdvice**. Esto evita try-catch dispersos y garantiza respuestas HTTP uniformes.

Clase	Extiende	Uso
NegocioException	RuntimeException	Violación de regla de negocio (ej: préstamo no PENDIENTE al intentar aprobar, pago <= 0). Retorna HTTP 400 Bad Request.
RecursoNoEncontradoException	RuntimeException	Entidad no encontrada en MongoDB (ej: cliente con ID inexistente). Retorna HTTP 404 Not Found.
GlobalExceptionHandler	@RestControllerAdvice	Captura NegocioException, RecursoNoEncontradoException y Exception genérica. Retorna un JSON {error, mensaje, timestamp}.

15. MÓDULO DE REPORTES

El servicio de reportes (**ReporteServiceImpl**) utiliza dos librerías externas para exportar datos en formatos distintos, demostrando el patrón Strategy.

Método	Descripción
generarReporteCartera(ReporteDTO)	Calcula totales: suma de montos, saldo pendiente total, cantidad de préstamos activos/cancelados. Retorna Map.
generarReporteMora(ReporteDTO)	Itera todos los préstamos y cuotas, invoca cuota.verificarMora(), y construye lista de cuotas vencidas con clienteld, numeroCuota, saldo y fechaVencimiento.
generarReportePagos(ReporteDTO)	Recorre la lista de pagos embebida en cada préstamo y construye una lista plana con prestamold, clienteld, valor, fecha y metodoPago.
exportarPDF(ReporteDTO) : byte[]	Usa iText (com.itextpdf) para construir un Document con tabla PdfPTable. Escribe en ByteArrayOutputStream y retorna los bytes para descarga.
exportarExcel(ReporteDTO) : byte[]	Usa Apache POI (XSSFWorkbook) para crear una hoja con encabezados y filas de datos. Escribe en ByteArrayOutputStream y retorna los bytes.

16. ENDPOINTS REST — RESUMEN COMPLETO

Método	Endpoint	Roles	Descripción
POST	/api/auth/login	Público	Autenticar y obtener JWT
POST	/api/auth/refresh	Público	Renovar access token
GET	/api/auth/logout	Autenticado	Cerrar sesión
PUT	/api/auth/cambiar-password	Autenticado	Cambiar contraseña
DELETE	/api/auth/fin	Autenticado	Finalizar sesión
POST	/api/clientes	ADM, REC	Registrar cliente
GET	/api/clientes	ADM, REC	Listar clientes
GET	/api/clientes/{id}	ADM, REC	Consultar cliente por ID
PUT	/api/clientes/{id}	ADM, REC	Actualizar cliente
DELETE	/api/clientes/{id}	ADM, REC	Eliminar cliente
POST	/api/prestamos	ADM, REC, CLI	Solicitar préstamo
GET	/api/prestamos	ADM, REC, CLI	Listar todos los préstamos
GET	/api/prestamos/{id}	ADM, REC, CLI	Consultar préstamo
GET	/api/prestamos/cliente/{id}	ADM, REC, CLI	Préstamos de un cliente
PUT	/api/prestamos/{id}/aprobar	ADM, REC	Aprobar préstamo
PUT	/api/prestamos/{id}/rechazar	ADM, REC	Rechazar préstamo
POST	/api/pagos	ADM, REC	Registrar pago
GET	/api/pagos/prestamo/{id}	ADM, REC	Pagos de un préstamo
GET	/api/pagos/prestamo/{pld}/comprobante/{pagoId}	ADM, REC	Obtener comprobante
GET	/api/cuotas/prestamo/{id}	ADM, REC, CLI	Cuotas de un préstamo
POST	/api/usuarios	ADM	Crear usuario
GET	/api/usuarios	ADM	Listar usuarios
GET	/api/usuarios/{id}	ADM	Consultar usuario
PUT	/api/usuarios/{id}	ADM	Actualizar usuario
PUT	/api/usuarios/{id}/estado	ADM	Cambiar estado
PUT	/api/usuarios/{id}/rol	ADM	Asignar rol
DELETE	/api/usuarios/{id}	ADM	Desactivar usuario
GET	/api/reportes/cartera	ADM	Reporte de cartera
GET	/api/reportes/mora	ADM	Reporte de mora
GET	/api/reportes/pagos	ADM	Reporte de pagos
GET	/api/reportes/exportar/pdf	ADM	Exportar PDF
GET	/api/reportes/exportar/excel	ADM	Exportar Excel
GET	/api/parametros	ADM	Listar parámetros
GET	/api/parametros/{clave}	ADM	Consultar parámetro

PUT	/api/parametros/{id}	ADM	Actualizar parámetro
-----	----------------------	-----	----------------------

17. TECNOLOGÍAS Y DEPENDENCIAS

Tecnología / Librería	Uso en el proyecto
Java 17	Lenguaje de programación principal. Uso de lambdas, Stream API, Optional y records (para DTOs).
Spring Boot 3	Framework principal: auto-configuración, gestión de dependencias, servidor embebido Tomcat.
Spring Data MongoDB	Capa de persistencia: MongoRepository, @Document, @DBRef, @Id.
Spring Security 6	Seguridad: cadena de filtros, BCrypt, DaoAuthenticationProvider, SessionCreationPolicy.STATELESS.
JWT (io.jsonwebtoken)	Librería para generar, firmar y validar tokens JWT con HMAC-SHA256.
Lombok	Reducción de boilerplate: @Data, @NoArgsConstructor, @AllArgsConstructor, @EqualsAndHashCode.
iText (com.itextpdf)	Generación de documentos PDF desde código Java (reportes de cartera).
Apache POI	Generación de archivos Excel (.xlsx) con XSSFWorkbook (reportes tabulares).
MongoDB	Base de datos NoSQL orientada a documentos BSON. Colecciones: clientes, prestamos, usuarios, roles, parametros.
Maven	Gestión de dependencias y ciclo de vida del proyecto (pom.xml).

18. CONCLUSIONES

Arquitectura limpia y escalable

La separación estricta en capas (Controller → Service → Repository → Domain) garantiza bajo acoplamiento y alta cohesión. Cada capa puede modificarse o reemplazarse de forma independiente.

Herencia bien aplicada

La jerarquía Usuario → Administrador / Recaudador / ClienteUsuario demuestra el principio de reutilización de código. Los atributos y métodos comunes están en la clase base; las subclasses solo añaden su comportamiento específico.

Polimorfismo a través de interfaces

Todos los servicios exponen interfaces; Spring inyecta la implementación concreta. Esto facilita las pruebas unitarias (se puede inyectar un mock) y permite cambiar la implementación sin tocar el controlador.

Seguridad robusta

El sistema usa JWT stateless con BCrypt, controlando el acceso por roles en cada endpoint. No almacena sesiones en servidor, facilitando el escalado horizontal.

Integridad financiera

El uso exclusivo de BigDecimal con RoundingMode.HALF_UP para todos los cálculos monetarios garantiza precisión en operaciones críticas como cuotas, intereses y mora.

Documentos embebidos estratégicos

Cuota y Pago embebidos en Prestamo reducen las operaciones JOIN/lookup y aseguran consistencia transaccional en una sola operación de escritura MongoDB.

Extensibilidad del módulo de reportes

El ReporteService puede extenderse para soportar CSV, JSON o cualquier otro formato sin modificar el controlador, siguiendo el principio abierto/cerrado.

Información del documento

Sistema: Sistema de Recaudo de Préstamos de Dinero

Autor: Camilo Andrés Castro Acosta

Generado: 03/06/2026 02:41

Tecnología: Java 17 · Spring Boot 3 · MongoDB · JWT · iText · Apache POI